

课程笔记

CS146S Week 7: Modern Software Support (Code Review)

基于 Mihail Eric + Tomas Reimers (Graphite CPO) 授课内容整理

2025 年 11 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 1 lecture (Mon)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 代码审查的价值与方法	2
1.1 为什么代码审查是最高杠杆的工程活动	2
1.2 代码审查应该关注什么	2
1.2.1 逻辑与正确性错误	2
1.2.2 可读性与可维护性	2
1.2.3 性能	2
1.2.4 安全	2
1.2.5 最佳实践	2
1.3 什么是好的代码审查	2
1.4 本章小结	3
2 把代码审查设计成一个系统	3
2.1 审查质量首先取决于 PR 质量	3
2.2 审查意见的三层结构	3
2.3 本章小结	3
3 AI 代码审查的新世界	3
3.1 主要工具	3
3.2 AI 代码审查带来的改变	4
3.3 当前的局限	4
3.4 AI 审查系统的落地架构	4
3.5 人类 reviewer 在哪里最不可替代	5
3.6 团队应该怎样衡量 review 系统是否真的变好	5
3.7 本章小结	6
4 总结与延伸	6
4.1 关键点	6
4.2 团队落地建议	6
4.3 审查策略对照表	7
4.4 拓展阅读	7

1 代码审查的价值与方法

1.1 为什么代码审查是最高杠杆的工程活动

代码审查 (Code Review) 是提升个人工程能力和团队代码质量最具杠杆效应的活动之一。数据佐证：

代码审查的量化价值

- 代码审查的错误检出率为 55–60%，而各种测试模式的检出率仅为 25–45%
- 无代码审查 vs 有代码审查的程序错误率：4.5 → 0.82 errors/100 lines
- AT&T 的研究显示：代码审查带来 14% 的生产力提升和 90% 的缺陷减少

来源：Coding Horror

1.2 代码审查应该关注什么

1.2.1 逻辑与正确性错误

代码是否按预期工作？是否有边界条件处理不当、off-by-one 错误等？

1.2.2 可读性与可维护性

代码是否易于理解和后续修改？命名是否清晰？函数职责是否单一？

1.2.3 性能

是否存在不必要的数据库查询、N+1 问题、不合理的时间/空间复杂度？

1.2.4 安全

是否存在 SQL injection、XSS、未验证的输入、敏感数据泄露等安全隐患？

1.2.5 最佳实践

是否遵循项目的编码规范、设计模式和架构约定？

1.3 什么是好的代码审查

“This won't work” 不是好的审查意见

简单的否定不提供任何教学价值。好的审查意见应该是建设性的、具体的，并邀请对话。例如：

“I see your new method matches the existing style in this file, taking [X] parameters. Having that many parameters hurts readability and implies the function is doing too much. What do you think about refactoring this method and the existing ones in a later pull request to reduce how many parameters they take?”

1.4 本章小结

代码审查是软件质量的核心保障，其错误检出率远超自动化测试。好的审查意见应该是具体的、建设性的，并为被审查者提供学习机会。

2 把代码审查设计成一个系统

2.1 审查质量首先取决于 PR 质量

课程里一个非常实际的观点是：很多所谓“低质量审查”，根源并不在 reviewer，而在 pull request 本身过大、上下文不足、变更意图不清晰。要让审查有效，团队需要先把 PR 设计成可审查对象。

- 控制单个 PR 的规模，让 reviewer 能在有限时间内建立完整心智模型
- 提供变更背景：问题是什么、为什么现在改、是否有替代方案
- 写清验证方法：跑了哪些测试、有哪些已知风险
- 对大改动做堆叠（stacked diffs）或按主题拆分，而不是一次性提交

审查效率不是 reviewer 一个人的 KPI

如果作者提交的是一个 3000 行、跨十个模块、没有说明的 PR，那么再强的 reviewer 也只能给出表层反馈。高效审查是作者、工具和 reviewer 共同设计出来的流程能力。

2.2 审查意见的三层结构

一条好的 review comment 通常可以拆成三层：

1. **观察**：指出具体代码或行为
2. **原因**：解释为什么这会造成风险、复杂度或歧义
3. **建议**：给出替代方案、提问路径或后续改进方向

从“挑错”到“共同设计”

资深团队往往把 code review 看成设计讨论的最后一道关口，而不是上线前的抱怨区。评论最有价值的部分，通常不是指出 bug，而是帮助作者看到设计上的长期成本。

2.3 本章小结

代码审查要想高质量，前提是 PR 本身可被审查。小而清晰的变更、明确的背景和验证信息，能把审查从机械 diff 浏览升级为真正的工程讨论。

3 AI 代码审查的新世界

3.1 主要工具

- **Graphite**：本周嘉宾 Tomas Reimers 是其 CPO

- **Greptile**: 基于代码库理解的 AI 审查
- **CodeRabbit**: 自动化 PR 审查
- **Claude Code / Codex**: 通用 AI 编码工具也可用于审查

3.2 AI 代码审查带来的改变

1. **效率提升**: 自动化处理常规检查, 缩短审查周期
2. **一致性**: AI 对所有代码应用相同的标准
3. **知识共享**: AI 可以携带整个代码库的上下文
4. **减轻认知负担**: 让人类专注于复杂逻辑和架构决策
5. **持续改进**: AI 系统随时间积累更多数据而改善
6. **整体理解**: AI 可以从全局视角审视变更的影响

AI 代码审查的分工模式

最佳实践不是用 AI 完全替代人类审查, 而是让 AI 处理“表层”问题 (格式、命名、常见 bug 模式、安全检查), 让人类专注于“深层”问题 (架构合理性、业务逻辑正确性、长期维护性)。

3.3 当前的局限

- **配置和设置成本**: 需要初始投入来训练系统
- **假阳性**: 需要持续训练系统以减少误报
- **无法捕获代码库的惯用法和最佳实践**: 仓库特有的 idiom 仍然超出 AI 能力
- **无法处理复杂的业务逻辑和架构决策**: 这正是人类仍然不可或缺的地方
- **安全变更需格外谨慎**: AI 可能遗漏微妙的安全影响
- **经常遗漏边界情况**

AI 时代代码审查更重要了, 而不是更不重要

当 AI 编写了你的大部分代码时, 代码审查变得**更加**重要。你拥有被合并和部署的代码——不能把锅甩给 AI。“The AI wrote it” 不是一个可接受的借口。

3.4 AI 审查系统的落地架构

如果把 AI review 看成一个工程系统, 而不是一个 chatbot, 它至少包含以下流水线:

阶段	关键输入	主要任务
变更解析	Git diff、文件类型、提交信息	判断应该关注哪些模块、哪些变更具有高风险
仓库检索	相关文件、历史 PR、代码规范	补足 diff 之外的上下文，减少脱离语境的评论
规则执行	Lint、安全规则、团队约束	过滤显而易见的问题，降低人工负担
语义评估	逻辑流、接口契约、异常路径	发现更深层的正确性或维护性风险
评论生成	风险等级、证据、建议方案	输出可执行、可讨论、可归因的 review comments

表 1: AI 代码审查的典型流水线

最大的失败模式是上下文装配错误

很多 AI review 产品的问题并不是模型不会写评论，而是没有把足够的代码库上下文、团队规则和历史惯例喂给模型。上下文一旦错位，就会产生大量貌似合理但实际上没用的评论。

3.5 人类 reviewer 在哪里最不可替代

课程和行业实践都指向同一个结论：AI 越擅长检查局部模式，人类 reviewer 越应该把时间投入到高阶判断。

- **意图校验**：这次改动真的解决了正确的问题吗？
- **架构一致性**：它是否让系统朝更混乱的方向演化？
- **组织记忆**：是否违背了团队过往踩过坑后形成的隐性规则？
- **风险取舍**：某些缺陷是否可以接受，某些技术债是否应该延期？

未来的 reviewer 更像“最后签字的系统设计师”

在 AI 生成代码的比例越来越高时，reviewer 不只是看格式或命名的人，而是对变更是否应该进入主干负责的人。这个角色要求更强的系统理解，而不是更快地扫 diff。

3.6 团队应该怎样衡量 review 系统是否真的变好

如果没有度量，团队往往会把“评论数量变多”误判为“审查质量变高”。更合理的指标应同时覆盖速度、质量和信任。

- **首轮反馈时间**：作者提交后多久收到第一批有效评论
- **缺陷逃逸率**：被合并后仍流入生产环境的问题比例
- **假阳性率**：AI 评论中被证明无效的比例
- **重开 PR 比例**：说明前期审查是否真正发现核心问题

- **reviewer 负担**：资深工程师在低价值评论上消耗了多少时间

好的审查系统不是让每个 PR 更吵，而是让高风险 PR 更清晰

如果 AI 给每个 diff 都生成大量模板化评论，团队很快就会形成忽视习惯。真正有效的系统应该把注意力集中在最关键的问题上，让重要评论更容易被认真处理。

3.7 本章小结

AI 代码审查工具通过自动化常规检查显著提升了效率和一致性，但复杂的业务逻辑、架构决策和仓库特有的最佳实践仍然需要人类审查者。在 AI 编码时代，代码审查的重要性不降反升。

4 总结与延伸

Week 7 探讨了代码审查在 AI 编码时代的角色演变。核心信息是：AI 让代码审查更高效，但同时也让代码审查更加不可或缺——因为你需要审查的不仅是人写的代码，还有 AI 写的代码。

4.1 关键点

- 代码审查的错误检出率（55–60%）远超测试（25–45%）
- 好的审查意见是建设性的、具体的，并邀请对话
- 审查应覆盖五个维度：正确性、可读性、性能、安全、最佳实践
- AI 代码审查适合处理“表层”问题，人类专注“深层”问题
- 当前局限：假阳性、无法捕获 repo-specific idioms、无法处理复杂业务逻辑
- AI 时代代码审查更重要，“AI wrote it”不是免责借口

4.2 团队落地建议

1. 先把 PR 模板、变更说明和测试说明标准化，再引入 AI review
2. 让 AI 先覆盖高频表层问题，不要一开始就要求它替代资深 reviewer
3. 为关键评论建立证据链：引用 diff、规则或历史上下文
4. 持续统计假阳性与漏报率，把 review 系统当产品运营
5. 在安全、支付、权限等高风险模块保留更强的人类审批门槛

4.3 审查策略对照表

场景	更适合 AI 的任务	更适合人类的任务
常规业务 PR	命名、重复逻辑、常见 bug、规则检查	业务意图、边界取舍、代码演进方向
基础设施改造	配置校验、依赖影响扫描	回滚策略、兼容性风险、迁移节奏
安全敏感模块	已知漏洞模式匹配、权限变更提示	威胁建模、攻击面判断、例外审批
大规模重构	模式一致性检查、遗漏引用发现	是否值得重构、拆分策略、上线窗口

表 2: AI reviewer 与人类 reviewer 的分工边界

4.4 拓展阅读

- Graphite: <https://graphite.dev>
- Greptile: <https://greptile.com>
- CodeRabbit: <https://coderabbit.ai>
- Google 的代码审查最佳实践: <https://google.github.io/eng-practices/review/>
- “How to do a code review” by Blake Smith